

React & TypeScript fundamentals: exercises

Learner practice for Lectures 18 and 19

These exercises cover the React concepts introduced in Lectures 18 and 19.

For coding exercises, use React with TypeScript. Unless an exercise asks you to create a new project, you may use an existing React TypeScript practice project.

The focus is on:

- React project setup and development tooling
- TSX
- Function components
- Typed props
- Prop destructuring
- Component reuse
- Reading basic class components

Do not use concepts from later React lessons, such as state, hooks, event handlers, `.map()`, routing, or API calls.

Section A: Project setup and tooling

Exercise 1: Set up an existing project

You receive this React TypeScript project from another developer:

```
conference-board/  
├── public/  
├── src/  
│   ├── App.tsx  
│   └── main.tsx  
├── index.html  
├── package.json  
├── package-lock.json  
└── vite.config.ts
```

There is no `node_modules` folder.

The project has not yet been run on your computer.

Answer the following:

1. Does the missing `node_modules` folder mean that the project is broken? Explain briefly.
2. What command should you run before trying to start the project?
3. Which file would you inspect to see the dependencies declared by the project?
4. What folder should appear after the dependencies have been installed?
5. Why is copying another developer's `node_modules` folder not the normal way to set up the project?

Exercise 2: Reconstruct the React project setup

You want to create a React TypeScript project named:

```
react-practice
```

The following actions are out of order:

```
Open the local URL shown in the terminal
```

```
Install the project dependencies
```

```
Start the development server
Enter the react-practice folder
Create the Vite React TypeScript project
```

Part A

Put the five actions in the correct order.

Part B

Write the terminal command you would use for each command-line step.

Use the same Vite React TypeScript setup used during the lecture.

Part C

In one sentence each, explain the role of:

- Node.js
- npm
- Vite

Part D

You want to run a command-line tool provided by an npm package without first installing that tool globally.

Explain how npx can be useful in this situation.

Section B: Working with TSX

Exercise 3: Repair the returned TSX

The following component does not compile:

```
function BuildSummary() {
  const pendingChecks = 4;

  return (
    <h1>Build summary</h1>

    <p>
      Pending checks: {pendingChecks}
    </p>
  );
}
```

Correct the component so that:

- both visible elements remain,
- the value of pendingChecks is still rendered through a TSX expression,
- the component returns valid TSX.

After making the correction, briefly explain what was wrong with the original return value.

Exercise 4: Convert HTML into TSX

You have these values inside a React component:

```
const productName = "Mechanical keyboard";
const unitsAvailable = 14;
```

The original HTML design is:

```
<section class="product-summary">
  <h2>Mechanical keyboard</h2>
```

```
<p>14 units available</p>
</section>
```

Rewrite this markup as valid TSX.

Your TSX must:

- preserve the product-summary CSS class,
- display productName inside the heading,
- display unitsAvailable inside the paragraph,
- keep units available as normal text,
- use the variables rather than typing "Mechanical keyboard" or 14 directly into the returned markup.

Section C: Components and typed props

Exercise 5: Trace props through a component

Read the following code:

```
type WorkshopSummaryProps = {
  topic: string;
  room: string;
  duration: number;
};

function WorkshopSummary({
  topic,
  room,
  duration,
}: WorkshopSummaryProps) {
  return (
    <section>
      <h2>{topic}</h2>
      <p>Room: {room}</p>
      <p>Duration: {duration} min</p>
    </section>
  );
}

function App() {
  return (
    <main>
      <WorkshopSummary
        topic="Testing React interfaces"
        room="Lab 4"
        duration={50}
      />
    </main>
  );
}
```

Answer the following without changing the code:

1. Which component supplies the values for topic, room, and duration?
2. Which component receives those values?
3. Which variable contains "Lab 4" inside WorkshopSummary?
4. What value does duration contain inside WorkshopSummary?
5. Write the visible text produced by WorkshopSummary, in the order it appears.

Exercise 6: Repair a prop mismatch

A component expects this prop shape:

```
type ProductSummaryProps = {
  name: string;
  stock: number;
};
```

It is currently used like this:

```
<ProductSummary
  title="Wireless mouse"
  stock={7}
/>
```

Compare the component use with ProductSummaryProps.

Then:

1. Identify the mismatch.
 2. Rewrite the component use so that it satisfies the prop type.
 3. Briefly explain what TypeScript is checking in this example.
- Keep the supplied ProductSummaryProps definition unchanged.

Exercise 7: Add typed props to a component

The following component reads values from props, but its props have not yet been typed:

```
function VenueInfo(props) {
  return (
    <section>
      <h2>{props.name}</h2>
      <p>Capacity: {props.capacity}</p>
    </section>
  );
}
```

VenueInfo should receive:

Prop	Type
name	string
capacity	number

Complete the component so that:

1. A TypeScript type called VenueInfoProps describes the expected props.
2. The component parameter is typed using VenueInfoProps.
3. The existing rendered content remains unchanged.

Then render VenueInfo once from App using:

```
Name: Innovation Hall
Capacity: 180
```

Supply 180 as a number.

Exercise 8: Refactor props using destructuring

The following component already works:

```
type BookSummaryProps = {
  title: string;
  author: string;
  pages: number;
};
```

```
function BookSummary(
  props: BookSummaryProps
) {
  return (
    <article>
      <h2>{props.title}</h2>
      <p>{props.author}</p>
      <p>{props.pages} pages</p>
    </article>
  );
}
```

Refactor BookSummary so that:

- title, author, and pages are destructured in the function parameter,
- the JSX uses the destructured variables directly,
- BookSummaryProps remains unchanged,
- the rendered result remains unchanged.

Then answer:

After this refactor, has the component received different data, or has only the way it reads the props changed?

Explain briefly.

Exercise 9: Refactor repeated JSX into a component

The following App works, but the same UI structure is repeated:

```
function App() {
  return (
    <main>
      <h1>Project overview</h1>

      <section>
        <div>
          <strong>12</strong>
          <span>Open issues</span>
        </div>

        <div>
          <strong>7</strong>
          <span>Reviews pending</span>
        </div>

        <div>
          <strong>3</strong>
          <span>Deployments today</span>
        </div>
      </section>
    </main>
  );
}
```

Refactor the repeated <div> structure into one reusable function component.

Your solution must:

1. Identify the values that differ between the three repeated blocks.
2. Create an appropriate TypeScript prop type for those values.
3. Define one reusable component with a capitalised name.
4. Destructure its props.
5. Render the component three times from App.
6. Preserve the following data:

Value	Label
12	Open issues
7	Reviews pending
3	Deployments today

Do not create separate component definitions for the three items.

Section D: Function and class components

Exercise 10: Compare two component forms

The following components are intended to display the same kind of information.

Function component

```
type StatusLabelProps = {
  text: string;
};

function StatusLabel({
  text,
}: StatusLabelProps) {
  return <span>{text}</span>;
}
```

Class component

```
import { Component } from "react";

type StatusLabelProps = {
  text: string;
};

class StatusLabelClass
  extends Component<StatusLabelProps> {
  render() {
    return (
      <span>{this.props.text}</span>
    );
  }
}
```

Answer the following:

1. Where does the function component receive its props?
2. How does the class component access its props?
3. Where does the function component return its JSX?
4. Which method returns JSX in the class component?
5. What does `Component<StatusLabelProps>` tell TypeScript?
6. If both components receive the text "Available", can they produce the same visible result? Explain briefly.

Final practical: Riverfront Sessions

You are building the first version of a small festival schedule using React and TypeScript.

The application does not need any interaction. Your goal is to create a component-based interface, pass typed data through props, and reuse a component for several performances.

No additional CSS is required, and styling is not assessed.

Part 1: Create and run the project

Create a Vite React TypeScript project named:

```
riverfront-schedule
```

Use:

```
npm create vite@latest riverfront-schedule -- --template react-ts --no-interactive
```

Then:

```
cd riverfront-schedule
npm install
npm run dev
```

Open the local URL shown in the terminal and confirm that the starter application runs before continuing.

Part 2: Create the components

Inside src, create a components folder containing:

```
src/
├── components/
│   ├── FestivalHeader.tsx
│   ├── PerformanceRow.tsx
│   └── FestivalFooter.tsx
└── App.tsx
```

Each component should be defined in its own file.

Export each component and import it where it is used. You may use either named or default exports.

Part 3: Create FestivalHeader

Create a function component named FestivalHeader.

It must receive these props:

Prop	Type
festivalName	string
day	string
openingTime	string

Define an appropriate TypeScript props type or interface.

Use the component from App with these values:

```
festivalName: Riverfront Sessions
day: Saturday
openingTime: 4:00 PM
```

The component should display:

Riverfront Sessions
 Saturday
 Gates open at 4:00 PM

openingTime should contain only "4:00 PM". The words "Gates open at" belong to the component's rendered TSX.

Part 4: Create PerformanceRow

Create one reusable function component named PerformanceRow.

It must receive:

Prop	Type
artist	string
startTime	string
duration	number

Requirements:

- Define an appropriate TypeScript props type or interface.
- Type the component using that prop definition.
- Destructure artist, startTime, and duration in the component parameter.
- Render all three values.
- Display the duration followed by min.

For example, if duration receives the number 45, the rendered text should include:

45 min

Part 5: Render the performances

Use the same PerformanceRow component for all four performances:

Start time	Artist	Duration
4:30 PM	Paper Satellites	45
5:30 PM	Glass Harbour	50
6:45 PM	The Midnight Archive	60
8:15 PM	Northern Lights	75

Write the four component instances directly in your TSX.

Do not use .map() for this exercise.

Part 6: Create FestivalFooter

Create a function component named FestivalFooter.

This component does not need props.

It should display:

Venue: Riverside Amphitheatre

Part 7: Compose the interface in App

Use App to assemble the page.

The interface must contain, in this order:

1. One FestivalHeader
2. A heading for the schedule
3. Four PerformanceRow instances
4. One FestivalFooter

Choose suitable HTML elements for the page structure.

The assessment focuses on your React component structure, JSX, TypeScript props, data flow, and component reuse, not on a particular choice of HTML tags or visual styling.

Expected content

When the project is complete, the browser should contain this information:

```
Riverfront Sessions
Saturday
Gates open at 4:00 PM

Schedule

4:30 PM
Paper Satellites
45 min

5:30 PM
Glass Harbour
50 min

6:45 PM
The Midnight Archive
60 min

8:15 PM
Northern Lights
75 min

Venue: Riverside Amphitheatre
```

The exact visual arrangement does not need to match this text layout.

Completion check

- The project runs successfully.
- There are no unintended TypeScript errors.
- FestivalHeader, PerformanceRow, and FestivalFooter are separate components.
- Each component is exported from its own file.
- The required props are typed.
- PerformanceRow destructures its props.
- duration is a number.
- PerformanceRow is defined once and rendered four times.
- Each performance supplies its own prop values.
- All required festival information appears in the browser.
- FestivalFooter works without props.
- App composes the components into the finished interface.
- No state, hooks, event handlers, .map(), routing, or API calls have been added.